基于这份详细的 API 文档，我来为你展示如何系统地使用这些接口进行前后端开发。以下是完整的使用指南：

1. Postman 配置和使用

创建 Postman Collection

```
{
  "info": {
    "name": "XMU动物友伴 API",
    "description": "厦门大学校园动物管理系统 API",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    {
      "key": "base_url",
      "value": "http://localhost:8080",
      "type": "string"
    },
    {
      "key": "token",
      "value": "",
      "type": "string"
    }
  ],
  "item": [
    {
      "name": "用户登录",
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "value": "application/json"
          }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\n  \"schoolId\": \"2123001\",\n  \"username\": \"张三\"\n}"
        },
        "url": {
          "raw": "{{base_url}}/api/auth/login",
          "host": ["{{base_url}}"],
          "path": ["api", "auth", "login"]
        }
      },
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "// 保存 token 到环境变量",
              "var jsonData = pm.response.json();",
```

```
                "if (jsonData.code === 200) {",
                "    pm.environment.set('token', jsonData.data.token);",
                "    console.log('Token saved:', jsonData.data.token);",
                "}"
              ]
            }
          }
        ]
      }
    ]
}
```

环境变量配置

创建两个环境：

- **开发环境**：`base_url = http://localhost:8080`
- **生产环境**：`base_url = https://api.xmu-animal.edu.cn`

2. Vue 前端实现

项目结构和依赖

```
# 创建 Vue 项目
vue create xmu-animal-frontend
cd xmu-animal-frontend

# 安装必要依赖
npm install axios vue-router vuex element-plus
```

API 服务层封装

```
// src/api/index.js
import axios from 'axios'
import { Message } from 'element-ui'

// 创建 axios 实例
const service = axios.create({
  baseURL: process.env.VUE_APP_BASE_API || 'http://localhost:8080',
  timeout: 10000
})

// 请求拦截器 - 添加 token
service.interceptors.request.use(
  config => {
    const token = localStorage.getItem('token')
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`
    }
    return config
  },
  error => {
    return Promise.reject(error)
  }
)
```

```

// 响应拦截器 - 统一错误处理
service.interceptors.response.use(
  response => {
    const res = response.data
    if (res.code !== 200) {
      Message.error(res.message || '请求失败')
      return Promise.reject(new Error(res.message || 'Error'))
    }
    return res
  },
  error => {
    if (error.response?.status === 401) {
      Message.error('登录已过期，请重新登录')
      localStorage.removeItem('token')
      window.location.href = '/login'
    } else {
      Message.error(error.message || '网络错误')
    }
    return Promise.reject(error)
  }
)

export default service
```

具体 API 接口

```
// src/api/auth.js
import request from './index'

export const authApi = {
  // 用户登录
  login(data) {
    return request({
      url: '/api/auth/login',
      method: 'post',
      data
    })
  }
}

// src/api/animals.js
export const animalsApi = {
  // 获取动物列表
  getAnimals(params) {
    return request({
      url: '/api/animals',
      method: 'get',
      params
    })
  },

  // 获取动物详情
  getAnimalDetail(id) {
    return request({
```

```
      url: `/api/animals/${id}`,
      method: 'get'
    })
  },

  // 创建动物（管理员）
  createAnimal(data) {
    return request({
      url: '/api/animals',
      method: 'post',
      data
    })
  }
}

// src/api/sightings.js
export const sightingsApi = {
  // 提交目击记录
  submitSighting(data) {
    return request({
      url: '/api/sightings',
      method: 'post',
      data
    })
  },

  // 获取动态时间线
  getSightings(params) {
    return request({
      url: '/api/sightings',
      method: 'get',
      params
    })
  },

  // 审核目击记录（管理员）
  approveSighting(id) {
    return request({
      url: `/api/sightings/${id}/approve`,
      method: 'patch'
    })
  }
}

// src/api/upload.js
export const uploadApi = {
  // 上传图片
  uploadImage(file) {
    const formData = new FormData()
    formData.append('file', file)

    return request({
      url: '/api/upload',
      method: 'post',
```

```
      data: formData,
      headers: {
        'Content-Type': 'multipart/form-data'
      }
    })
  }
}
```

Vue 组件示例

```
<template>
  <div class="animal-app">
    <!-- 登录组件 -->
    <login-form v-if="!isLoggedIn" @login-success="handleLoginSuccess" />

    <!-- 主界面 -->
    <div v-else>
      <nav-bar :user="user" @logout="handleLogout" />

      <!-- 动物列表页面 -->
      <animal-list
        :animals="animals"
        :loading="loading"
        @filter-change="handleFilterChange"
        @view-detail="viewAnimalDetail"
      />

      <!-- 提交目击记录对话框 -->
      <sighting-dialog
        v-model="showSightingDialog"
        :animal-id="selectedAnimalId"
        @submit="handleSubmitSighting"
      />
    </div>
  </div>
</template>

<script>
import { authApi, animalsApi, sightingsApi } from '@/api'
import LoginForm from '@/components/LoginForm.vue'
import NavBar from '@/components/NavBar.vue'
import AnimalList from '@/components/AnimalList.vue'
import SightingDialog from '@/components/SightingDialog.vue'

export default {
  name: 'AnimalApp',
  components: {
    LoginForm,
    NavBar,
    AnimalList,
    SightingDialog
  },
  data() {
    return {
      isLoggedIn: false,
```

```
46        user: null,
47        animals: [],
48        loading: false,
49        showSightingDialog: false,
50        selectedAnimalId: null,
51        filters: {
52          species: '',
53          area: '',
54          healthStatus: ''
55        }
56      }
57    },
58    methods: {
59      // 登录处理
60      async handleLoginSuccess(loginData) {
61        const { data } = await authApi.login(loginData)
62        localStorage.setItem('token', data.token)
63        this.user = data.user
64        this.isLoggedIn = true
65        this.loadAnimals()
66      },
67
68      // 加载动物列表
69      async loadAnimals() {
70        this.loading = true
71        try {
72          const { data } = await animalsApi.getAnimals(this.filters)
73          this.animals = data.content
74        } catch (error) {
75          console.error('加载动物列表失败:', error)
76        } finally {
77          this.loading = false
78        }
79      },
80
81      // 筛选条件变化
82      handleFilterChange(filters) {
83        this.filters = { ...filters }
84        this.loadAnimals()
85      },
86
87      // 提交目击记录
88      async handleSubmitSighting(sightingData) {
89        try {
90          await sightingsApi.submitSighting(sightingData)
91          this.$message.success('目击记录提交成功，等待审核')
92          this.showSightingDialog = false
93        } catch (error) {
94          console.error('提交目击记录失败:', error)
95        }
96      },
97
98      handleLogout() {
99        localStorage.removeItem('token')
```

```
      this.isLoggedIn = false
      this.user = null
      this.animals = []
    }
  },
  mounted() {
    // 检查本地是否有登录状态
    const token = localStorage.getItem('token')
    if (token) {
      // 这里可以添加 token 验证接口
      this.isLoggedIn = true
      this.loadAnimals()
    }
  }
}
</script>
```

环境配置

```
// .env.development
VUE_APP_BASE_API=http://localhost:8080
VUE_APP_TITLE=XMU动物友伴-开发版

// .env.production
VUE_APP_BASE_API=https://api.xmu-animal.edu.cn
VUE_APP_TITLE=XMU动物友伴
```

3. Spring Boot 后端实现

Maven 依赖

```
<!-- pom.xml -->
<dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- 数据库 -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <scope>runtime</scope>
    </dependency>

    <!-- JWT -->
```

```
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.5</version>
    </dependency>

    <!-- 文档 -->
    <dependency>
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.3.0</version>
    </dependency>

    <!-- 工具 -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

实体类定义

```
// Animal 实体
@Entity
@Table(name = "animals")
@Data
public class Animal {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long animalId;

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Species species; // CAT, DOG, SWAN

    @Enumerated(EnumType.STRING)
    private Gender gender; // MALE, FEMALE

    private String area;
    private String appearance;
    private String personality;

    @Enumerated(EnumType.STRING)
    private HealthStatus healthStatus; // HEALTHY, INJURED, ADOPTABLE

    private Boolean isNeutered = false;
    private Boolean isMissing = false;

    private String avatarUrl;

    @ElementCollection
```

```java
    private List<String> photoUrls = new ArrayList<>();

    @ElementCollection
    private List<String> personalityTags = new ArrayList<>();

    private LocalDateTime lastSeenTime;

    @CreationTimestamp
    private LocalDateTime createTime;

    @UpdateTimestamp
    private LocalDateTime updateTime;
}

// Sighting 实体
@Entity
@Table(name = "sightings")
@Data
public class Sighting {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long sightingId;

    @ManyToOne
    @JoinColumn(name = "animal_id")
    private Animal animal;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    private String sightingLocation;
    private Double latitude;
    private Double longitude;
    private String description;

    @ElementCollection
    private List<String> imageUrls = new ArrayList<>();

    @Enumerated(EnumType.STRING)
    private SightingStatus status = SightingStatus.PENDING;

    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

Controller 实现示例

```java
@RestController
@RequestMapping("/api/animals")
@RequiredArgsConstructor
public class AnimalController {

    private final AnimalService animalService;

```

```java
    @GetMapping
    public ResponseEntity<Result<Page<Animal>>> getAnimals(
            @RequestParam(required = false) String species,
            @RequestParam(required = false) String area,
            @RequestParam(required = false) String healthStatus,
            @RequestParam(required = false) Boolean isMissing,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {

        AnimalQuery query = AnimalQuery.builder()
                .species(species)
                .area(area)
                .healthStatus(healthStatus)
                .isMissing(isMissing)
                .build();

        Page<Animal> animals = animalService.getAnimals(query, page, size);
        return ResponseEntity.ok(Result.success(animals));
    }

    @GetMapping("/{id}")
    public ResponseEntity<Result<Animal>> getAnimalDetail(@PathVariable Long id) {
        Animal animal = animalService.getAnimalById(id);
        return ResponseEntity.ok(Result.success(animal));
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Result<Animal>> createAnimal(@Valid @RequestBody CreateAnimalRequest request) {
        Animal animal = animalService.createAnimal(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(Result.success(animal));
    }
}
```

4. 开发工作流程

阶段1：API 测试和验证

1. 在 Postman 中导入 Collection
2. 配置环境变量
3. 按顺序测试接口：
    - 登录 → 获取 token
    - 获取动物列表
    - 提交目击记录
    - 测试上传功能

阶段2：前端开发

1. 搭建 Vue 项目结构
2. 实现 API 服务层
3. 开发页面组件
4. 集成路由和状态管理

阶段3：后端开发

1. 设计数据库表结构
2. 实现实体类和 Repository
3. 开发 Service 和 Controller
4. 配置安全和权限

阶段4：联调和部署

1. 前后端接口联调
2. 测试各种边界情况
3. 部署到测试环境
4. 生产环境部署

这样就能系统地基于这份 API 文档进行全栈开发了。需要我详细说明某个特定部分吗?

我来详细讲解阶段1的完整操作步骤:

1. 在 Postman 中导入 Collection

步骤1：创建 Postman Collection JSON 文件

首先创建 `XMU-Animal-API.postman_collection.json` 文件:

```
{
  "info": {
    "name": "XMU动物友伴 API",
    "description": "厦门大学校园动物管理系统 API Collection",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    {
      "key": "base_url",
      "value": "http://localhost:8080",
      "type": "string"
    },
    {
      "key": "token",
      "value": "",
      "type": "string"
    }
  ],
  "item": [
```

```
20      {
21        "name": "🔐 认证相关",
22        "item": [
23          {
24            "name": "用户登录",
25            "request": {
26              "method": "POST",
27              "header": [
28                {
29                  "key": "Content-Type",
30                  "value": "application/json"
31                }
32              ],
33              "body": {
34                "mode": "raw",
35                "raw": "{\n  \"schoolId\": \"2123001\",\n  \"username\": \"测
试用户\"\n}"
36              },
37              "url": {
38                "raw": "{{base_url}}/api/auth/login",
39                "host": ["{{base_url}}"],
40                "path": ["api", "auth", "login"]
41              }
42            },
43            "event": [
44              {
45                "listen": "test",
46                "script": {
47                  "exec": [
48                    "// 登录成功后自动保存 token",
49                    "if (pm.response.code === 200) {",
50                    "    const response = pm.response.json();",
51                    "    if (response.code === 200) {",
52                    "        pm.collectionVariables.set('token',
response.data.token);",
53                    "        console.log('Token 已保存:',
response.data.token);",
54                    "        pm.test('登录成功', function() {",
55                    "
pm.expect(response.data.token).to.be.a('string');",
56                    "
pm.expect(response.data.user).to.be.an('object');",
57                    "        });",
58                    "    }",
59                    "}"
60                  ]
61                }
62              }
63            ]
64          }
65        ]
66      },
67      {
68        "name": "🐱 动物档案",
```

```
69        "item": [
70          {
71            "name": "获取动物列表",
72            "request": {
73              "method": "GET",
74              "header": [
75                {
76                  "key": "Authorization",
77                  "value": "Bearer {{token}}"
78                }
79              ],
80              "url": {
81                "raw": "{{base_url}}/api/animals?species=cat&page=0&size=5",
82                "host": ["{{base_url}}"],
83                "path": ["api", "animals"],
84                "query": [
85                  {
86                    "key": "species",
87                    "value": "cat"
88                  },
89                  {
90                    "key": "page",
91                    "value": "0"
92                  },
93                  {
94                    "key": "size",
95                    "value": "5"
96                  }
97                ]
98              }
99            }
100         },
101         {
102           "name": "获取动物详情",
103           "request": {
104             "method": "GET",
105             "header": [
106               {
107                 "key": "Authorization",
108                 "value": "Bearer {{token}}"
109               }
110             ],
111             "url": {
112               "raw": "{{base_url}}/api/animals/1",
113               "host": ["{{base_url}}"],
114               "path": ["api", "animals", "1"]
115             }
116           }
117         }
118       ]
119     },
120     {
121       "name": "👀 目击记录",
122       "item": [
```

```
        {
          "name": "提交目击记录",
          "request": {
            "method": "POST",
            "header": [
              {
                "key": "Content-Type",
                "value": "application/json"
              },
              {
                "key": "Authorization",
                "value": "Bearer {{token}}"
              }
            ],
            "body": {
              "mode": "raw",
              "raw": "{\n  \"animalId\": 1,\n  \"sightingLocation\": \"图书馆东门台阶\",\n  \"latitude\": 24.4321,\n  \"longitude\": 118.1234,\n  \"description\": \"看到小橘在晒太阳，状态很好\",\n  \"imageUrls\": [\n  \"https://oss.xmu.edu.cn/sightings/test1.jpg\"\n  ]\n}"
            },
            "url": {
              "raw": "{{base_url}}/api/sightings",
              "host": ["{{base_url}}"],
              "path": ["api", "sightings"]
            }
          }
        },
        {
          "name": "获取动态时间线",
          "request": {
            "method": "GET",
            "header": [
              {
                "key": "Authorization",
                "value": "Bearer {{token}}"
              }
            ],
            "url": {
              "raw": "{{base_url}}/api/sightings?page=0&size=10",
              "host": ["{{base_url}}"],
              "path": ["api", "sightings"],
              "query": [
                {
                  "key": "page",
                  "value": "0"
                },
                {
                  "key": "size",
                  "value": "10"
                }
              ]
            }
          }
```

```
        }
      ]
    },
    {
      "name": "📤 文件上传",
      "item": [
        {
          "name": "上传图片",
          "request": {
            "method": "POST",
            "header": [
              {
                "key": "Authorization",
                "value": "Bearer {{token}}"
              }
            ],
            "body": {
              "mode": "formdata",
              "formdata": [
                {
                  "key": "file",
                  "type": "file",
                  "src": "/path/to/your/image.jpg"
                }
              ]
            },
            "url": {
              "raw": "{{base_url}}/api/upload",
              "host": ["{{base_url}}"],
              "path": ["api", "upload"]
            }
          }
        }
      ]
    }
  ]
}
```

步骤2：导入到 Postman

1. **打开 Postman**
2. **点击 Import 按钮**（左上角）
3. **选择导入方式**：
    - 拖拽 JSON 文件到窗口
    - 或者点击 "Upload Files" 选择文件
4. **确认导入**：检查导入的 Collection 结构

2. 配置环境变量

创建开发环境

1. **点击 Environments**（左上角眼睛图标）
2. **点击 "+" 创建新环境**
3. **填写环境信息**：

```
环境名称： XMU动物友伴-开发环境
变量：
- base_url: http://localhost:8080
- token: （留空，登录后自动填充）
```

4. **保存环境**
5. **选择该环境**（在环境下拉菜单中选择）

环境变量说明

变量名	初始值	用途	自动更新
`base_url`	`http://localhost:8080`	所有请求的基础URL	手动修改
`token`	空	JWT认证令牌	登录接口自动设置

3. 按顺序测试接口

第一步：用户登录 → 获取 token

操作步骤：

1. **打开 "用户登录" 请求**
2. **检查请求体**：

```json
{
  "schoolId": "2123001",
  "username": "测试用户"
}
```

3. **点击 Send 按钮**
4. **检查响应**：

预期成功响应：

```json
{
  "code": 200,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.xxxxx",
    "user": {
      "userId": 101,
      "username": "测试用户",
      "schoolId": "2123001",
      "role": "USER"
    }
  }
}
```

验证要点：

- ✅ HTTP 状态码: 200
- ✅ `code` 字段: 200
- ✅ `data.token` 存在且为字符串
- ✅ `data.user` 包含用户信息

检查环境变量：

- 查看右上角环境管理，确认 `token` 变量已自动更新

第二步：获取动物列表

操作步骤：

1. **打开 "获取动物列表" 请求**
2. **观察请求头**：自动包含 `Authorization: Bearer {{token}}`
3. **检查查询参数**：`species=cat&page=0&size=5`
4. **点击 Send 按钮**

预期成功响应：

```json
{
  "code": 200,
  "data": {
    "content": [
      {
        "animalId": 1,
        "name": "小橘",
        "species": "cat",
        "gender": "male",
        "area": "芙蓉湖",
        "avatarUrl": "https://oss.xmu.edu.cn/animals/xiaoju.jpg",
        "healthStatus": "healthy",
        "isMissing": false,
        "personalityTags": ["亲人", "爱晒太阳"]
      }
    ],
    "totalElements": 15,
    "totalPages": 3,
    "number": 0
  }
}
```

测试不同筛选条件：

测试场景	查询参数	预期结果
筛选猫咪	`species=cat`	只返回猫咪
筛选区域	`area=芙蓉湖`	只返回芙蓉湖的动物
健康状态	`healthStatus=healthy`	只返回健康动物
分页测试	`page=1&size=3`	返回第二页，每页3条

第三步：获取动物详情

操作步骤：

1. **打开 "获取动物详情" 请求**
2. **确认路径参数：** `/api/animals/1`
3. **点击 Send 按钮**

预期成功响应：

```
{
  "code": 200,
  "data": {
    "animalId": 1,
    "name": "小橘",
    "species": "cat",
    "gender": "male",
    "isNeutered": true,
    "area": "芙蓉湖",
    "appearance": "橘白相间，尾巴有黑色环纹",
    "personality": "性格温顺，喜欢被摸头",
    "healthStatus": "healthy",
    "isMissing": false,
    "avatarUrl": "https://oss.xmu.edu.cn/animals/xiaoju.jpg",
    "photoUrls": [
      "https://oss.xmu.edu.cn/animals/xiaoju1.jpg",
      "https://oss.xmu.edu.cn/animals/xiaoju2.jpg"
    ],
    "lastSeenTime": "2025-11-19T14:30:00"
  }
}
```

边界测试：

- 测试不存在的动物ID：`/api/animals/9999`
- 预期响应：404 错误

第四步：提交目击记录

操作步骤：

1. **打开 "提交目击记录" 请求**
2. **检查请求体：**

```
{
  "animalId": 1,
  "sightingLocation": "图书馆东门台阶",
  "latitude": 24.4321,
  "longitude": 118.1234,
  "description": "看到小橘在晒太阳，状态很好",
  "imageUrls": [
    "https://oss.xmu.edu.cn/sightings/test1.jpg"
  ]
}
```

3. **点击 Send 按钮**

预期成功响应：

```
{
  "code": 201,
  "data": {
    "sightingId": 1001
  }
}
```

验证要点：

- ✅ HTTP 状态码: 201 (Created)
- ✅ 返回新创建的目击记录ID

错误情况测试：

- 测试不存在的 `animalId`
- 测试缺少必填字段
- 测试坐标格式错误

第五步：获取动态时间线

操作步骤：

1. **打开 "获取动态时间线" 请求**
2. **点击 Send 按钮**

预期成功响应：

```
{
  "code": 200,
  "data": {
    "content": [
      {
        "sightingId": 1001,
        "animalName": "小橘",
        "userName": "测试用户",
        "sightingLocation": "图书馆东门台阶",
        "description": "看到小橘在晒太阳，状态很好",
        "imageUrls": ["https://oss.xmu.edu.cn/sightings/test1.jpg"],
        "createdAt": "2025-11-20T10:00:00"
      }
    ],
    "totalElements": 1,
    "totalPages": 1,
    "number": 0
  }
}
```

第六步：测试上传功能

准备工作：

- 准备测试图片文件 (jpg/png)
- 确认文件大小合理 (< 5MB)

操作步骤：

1. **打开 "上传图片" 请求**
2. **修改 Body 为 form-data**
3. **选择文件**:
    - Key: `file`
    - Type: `File`
    - Value: 点击选择文件，选择测试图片
4. **点击 Send 按钮**

预期成功响应：

```
{
  "code": 200,
  "data": "https://oss.xmu.edu.cn/sightings/abc123.jpg"
}
```

验证要点：

- ✅ 返回可访问的图片URL
- ✅ URL 格式正确

4. 完整测试流程检查表

✅ 认证流程

- [ ] 用户登录成功
- [ ] Token 自动保存到环境变量
- [ ] 后续请求自动携带 Token

✅ 动物相关接口

- [ ] 获取动物列表（带筛选）
- [ ] 获取动物详情
- [ ] 测试分页功能
- [ ] 测试筛选条件组合

✅ 目击记录流程

- [ ] 提交目击记录成功
- [ ] 获取动态时间线
- [ ] 验证目击记录状态（PENDING）

✅ 文件上传

- [ ] 图片上传成功
- [ ] 返回有效图片URL

✅ 错误处理

- [ ] 测试未授权访问 (401)
- [ ] 测试资源不存在 (404)
- [ ] 测试参数验证错误 (400)

5. 常见问题排查

问题1：登录失败

症状：返回 400 或 500 错误
解决：

- 检查请求体格式
- 确认 `schoolId` 不为空
- 检查后端服务是否启动

问题2：Token 无效

症状：返回 401 未授权
解决：

- 重新登录获取新 token
- 检查环境变量中的 token 是否正确
- 确认 token 未过期

问题3：上传失败

症状：返回 400 或 413 错误
解决：

- 检查文件大小
- 确认文件格式支持
- 检查 multipart/form-data 格式

完成以上所有测试后，API 接口就验证完成了，可以进入下一阶段的开发工作。

我来详细讲解阶段2的前端开发完整流程：

1. 搭建 Vue 项目结构

项目初始化

```
# 使用 Vue CLI 创建项目
vue create xmu-animal-frontend

# 选择配置
? Please pick a preset:
  Default ([Vue 2] babel, eslint)
  Default (Vue 3) ([Vue 3] babel, eslint)
❯ Manually select features

# 选择功能
? Check the features needed for your project:
 ◉ Babel
 ◯ TypeScript
 ◯ Progressive Web App (PWA) Support
 ◉ Router
 ◉ Vuex
 ◉ CSS Pre-processors
❯◉ Linter / Formatter
 ◯ Unit Testing
 ◯ E2E Testing

# 选择 Vue 版本
? Choose a version of Vue.js that you want to start the project with
  2.x
❯ 3.x

# 选择路由模式
? Use history mode for router? (Requires proper server setup for production fallback) Yes

# 选择 CSS 预处理器
? Pick a CSS pre-processor (PostCSS, Autoprefixer and CSS Modules are supported by default):
  Sass/SCSS (with dart-sass)
❯ Less
  Stylus

# 选择代码规范
? Pick a linter / formatter config:
  ESLint with error prevention only
  ESLint + Airbnb config
❯ ESLint + Standard config
  ESLint + Prettier

# 选择保存时检查
? Pick additional lint features:
  Lint on save
❯ Lint and fix on commit

# 配置文件位置
```

```
? Where do you prefer placing config for Babel, ESLint, etc.?
  In dedicated config files
❯ In package.json

# 保存为预设
? Save this as a preset for future projects? No
```

安装额外依赖

```bash
cd xmu-animal-frontend

# 安装 UI 框架和工具库
npm install element-plus axios vue-axios
npm install @element-plus/icons-vue  # Element Plus 图标
npm install dayjs                    # 日期处理
npm install nprogress                # 进度条

# 开发依赖
npm install --save-dev mockjs       # Mock 数据
```

项目目录结构

```
src/
├── api/                # API 接口
│   ├── index.js      # Axios 配置
│   ├── auth.js       # 认证相关
│   ├── animals.js    # 动物相关
│   ├── sightings.js  # 目击记录
│   ├── upload.js     # 文件上传
│   └── mock/         # Mock 数据
├── components/        # 公共组件
│   ├── common/       # 通用组件
│   └── business/     # 业务组件
├── views/            # 页面组件
│   ├── Home.vue
│   ├── Login.vue
│   ├── Animals.vue
│   ├── AnimalDetail.vue
│   ├── Sightings.vue
│   └── Profile.vue
├── router/           # 路由配置
│   └── index.js
├── store/            # 状态管理
│   ├── index.js
│   ├── modules/
│   └── types.js
├── utils/            # 工具函数
│   ├── request.js
│   ├── auth.js
│   └── common.js
├── assets/           # 静态资源
│   ├── images/
│   └── styles/
├── App.vue
└── main.js
```

配置文件

```
// vue.config.js
const { defineConfig } = require('@vue/cli-service')

module.exports = defineConfig({
  transpileDependencies: true,
  devServer: {
    port: 3000,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        pathRewrite: {
          '^/api': '/api'
        }
      }
    }
  },
  css: {
    loaderOptions: {
      less: {
        lessOptions: {
          modifyVars: {
            'primary-color': '#1890ff',
            'link-color': '#1890ff',
            'border-radius-base': '4px'
          },
          javascriptEnabled: true
        }
      }
    }
  }
})
```

```
// .env.development
VUE_APP_BASE_API=/api
VUE_APP_TITLE=XMU动物友伴-开发版
VUE_APP_MOCK=true
```

```
// .env.production
VUE_APP_BASE_API=https://api.xmu-animal.edu.cn
VUE_APP_TITLE=XMU动物友伴
VUE_APP_MOCK=false
```

2. 实现 API 服务层

Axios 全局配置

```
// src/utils/request.js
import axios from 'axios'
import { ElMessage } from 'element-plus'
import router from '@/router'

// 创建 axios 实例
```

```
7  const service = axios.create({
8    baseURL: process.env.VUE_APP_BASE_API,
9    timeout: 10000,
10   headers: {
11     'Content-Type': 'application/json;charset=utf-8'
12   }
13 })
14
15 // 请求拦截器
16 service.interceptors.request.use(
17   config => {
18     // 添加 token
19     const token = localStorage.getItem('token')
20     if (token) {
21       config.headers['Authorization'] = `Bearer ${token}`
22     }
23
24     // 显示加载提示
25     if (config.showLoading !== false) {
26       // 可以在这里添加全局 loading
27     }
28
29     return config
30   },
31   error => {
32     return Promise.reject(error)
33   }
34 )
35
36 // 响应拦截器
37 service.interceptors.response.use(
38   response => {
39     const res = response.data
40
41     // 业务状态码处理
42     if (res.code === 200 || res.code === 201) {
43       return res
44     } else {
45       // 业务错误
46       ElMessage.error(res.message || '请求失败')
47       return Promise.reject(new Error(res.message || 'Error'))
48     }
49   },
50   error => {
51     // HTTP 错误
52     if (error.response) {
53       const { status, data } = error.response
54
55       switch (status) {
56         case 400:
57           ElMessage.error(data?.message || '请求参数错误')
58           break
59         case 401:
60           ElMessage.error('登录已过期，请重新登录')
```

```
          localStorage.removeItem('token')
          localStorage.removeItem('user')
          router.push('/login')
          break
        case 403:
          ElMessage.error('权限不足')
          break
        case 404:
          ElMessage.error('请求资源不存在')
          break
        case 500:
          ElMessage.error('服务器内部错误')
          break
        default:
          ElMessage.error(error.message || '网络错误')
      }
    } else {
      ElMessage.error('网络连接失败，请检查网络')
    }

    return Promise.reject(error)
  }
)

export default service
```

API 接口模块化

```
// src/api/index.js
import service from '@/utils/request'

export default service
```

```
// src/api/auth.js
import request from './index'

export const authApi = {
  // 用户登录
  login(data) {
    return request({
      url: '/api/auth/login',
      method: 'post',
      data,
      showLoading: true
    })
  },

  // 退出登录
  logout() {
    return request({
      url: '/api/auth/logout',
      method: 'post'
    })
  },

```

```
  // 刷新 token
  refreshToken() {
    return request({
      url: '/api/auth/refresh',
      method: 'post'
    })
  }
}
```

```
// src/api/animals.js
import request from './index'

export const animalsApi = {
  // 获取动物列表
  getAnimals(params = {}) {
    return request({
      url: '/api/animals',
      method: 'get',
      params: {
        page: 0,
        size: 10,
        ...params
      }
    })
  },

  // 获取动物详情
  getAnimalDetail(id) {
    return request({
      url: `/api/animals/${id}`,
      method: 'get'
    })
  },

  // 创建动物（管理员）
  createAnimal(data) {
    return request({
      url: '/api/animals',
      method: 'post',
      data
    })
  },

  // 更新动物信息（管理员）
  updateAnimal(id, data) {
    return request({
      url: `/api/animals/${id}`,
      method: 'put',
      data
    })
  }
}
```

```
// src/api/sightings.js
import request from './index'
```

```

export const sightingsApi = {
  // 提交目击记录
  submitSighting(data) {
    return request({
      url: '/api/sightings',
      method: 'post',
      data
    })
  },

  // 获取动态时间线
  getSightings(params = {}) {
    return request({
      url: '/api/sightings',
      method: 'get',
      params: {
        page: 0,
        size: 10,
        ...params
      }
    })
  },

  // 审核目击记录（管理员）
  approveSighting(id) {
    return request({
      url: `/api/sightings/${id}/approve`,
      method: 'patch'
    })
  },

  // 获取我的目击记录
  getMySightings() {
    return request({
      url: '/api/user/sightings',
      method: 'get'
    })
  }
}
```

```
// src/api/upload.js
import request from './index'

export const uploadApi = {
  // 上传图片
  uploadImage(file) {
    const formData = new FormData()
    formData.append('file', file)

    return request({
      url: '/api/upload',
      method: 'post',
      data: formData,
      headers: {
```

```
        'Content-Type': 'multipart/form-data'
      },
      timeout: 30000 // 上传超时时间延长
    })
  },

  // 批量上传
  uploadImages(files) {
    const uploadPromises = files.map(file => this.uploadImage(file))
    return Promise.all(uploadPromises)
  }
}
```

```
// src/api/favorites.js
import request from './index'

export const favoritesApi = {
  // 收藏/取消收藏
  toggleFavorite(animalId) {
    return request({
      url: '/api/favorites',
      method: 'post',
      data: { animalId }
    })
  },

  // 获取我的收藏
  getMyFavorites() {
    return request({
      url: '/api/user/favorites',
      method: 'get'
    })
  },

  // 检查是否收藏
  checkFavorite(animalId) {
    return request({
      url: `/api/favorites/check/${animalId}`,
      method: 'get'
    })
  }
}
```

Mock 数据配置（开发阶段使用）

```
// src/api/mock/index.js
import Mock from 'mockjs'
import { authApi } from './auth'
import { animalsApi } from './animals'
import { sightingsApi } from './sightings'

// 判断是否开启 Mock
if (process.env.VUE_APP_MOCK === 'true') {
  // 模拟登录接口
  Mock.mock('/api/auth/login', 'post', authApi.login)
```

```

  // 模拟动物相关接口
  Mock.mock(/\/api\/animals/, 'get', animalsApi.getAnimals)
  Mock.mock(/\/api\/animals\/\d+/, 'get', animalsApi.getAnimalDetail)

  // 模拟目击记录接口
  Mock.mock('/api/sightings', 'post', sightingsApi.submitSighting)
  Mock.mock(/\/api\/sightings/, 'get', sightingsApi.getSightings)

  console.log('Mock 数据已启用')
}

export default Mock
```

```
// src/api/mock/auth.js
export const authApi = {
  login: function(options) {
    const { schoolId, username } = JSON.parse(options.body)

    if (!schoolId) {
      return {
        code: 400,
        message: '学号不能为空'
      }
    }

    return {
      code: 200,
      message: 'success',
      data: {
        token: Mock.Random.string(32),
        user: {
          userId: Mock.Random.integer(100, 999),
          username: username || '测试用户',
          schoolId: schoolId,
          role: 'USER'
        }
      }
    }
  }
}
```

3. 开发页面组件

主应用组件

```
<!-- src/App.vue -->
<template>
  <div id="app">
    <el-config-provider :locale="zhCn">
      <router-view />
    </el-config-provider>
  </div>
</template>

```

```
<script>
import { ElConfigProvider } from 'element-plus'
import zhCn from 'element-plus/es/locale/lang/zh-cn'

export default {
  name: 'App',
  components: {
    ElConfigProvider
  },
  data() {
    return {
      zhCn
    }
  }
}
</script>

<style>
#app {
  font-family: 'Helvetica Neue', Helvetica, 'PingFang SC', 'Hiragino Sans GB',
               'Microsoft YaHei', '微软雅黑', Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  color: #2c3e50;
  min-height: 100vh;
  background-color: #f5f5f5;
}

* {
  box-sizing: border-box;
}

.page-container {
  padding: 20px;
  max-width: 1200px;
  margin: 0 auto;
}
</style>
```

入口文件配置

```
// src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import store from './store'

// Element Plus
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'
import * as ElementPlusIconsVue from '@element-plus/icons-vue'

// 全局样式
import '@/assets/styles/index.less'
```

```

// Mock 数据（开发环境）
if (process.env.VUE_APP_MOCK === 'true') {
  require('@/api/mock')
}

const app = createApp(App)

// 注册 Element Plus 图标
for (const [key, component] of Object.entries(ElementPlusIconsVue)) {
  app.component(key, component)
}

app.use(store)
app.use(router)
app.use(ElementPlus)

app.mount('#app')
```

登录页面组件

```
<!-- src/views/Login.vue -->
<template>
  <div class="login-container">
    <div class="login-box">
      <div class="login-header">
        <h1>🐾 XMU动物友伴</h1>
        <p>登录到校园动物管理系统</p>
      </div>

      <el-form
        ref="loginFormRef"
        :model="loginForm"
        :rules="loginRules"
        class="login-form"
        @submit.prevent="handleLogin"
      >
        <el-form-item prop="schoolId">
          <el-input
            v-model="loginForm.schoolId"
            placeholder="请输入学号/工号"
            size="large"
            :prefix-icon="User"
          />
        </el-form-item>

        <el-form-item prop="username">
          <el-input
            v-model="loginForm.username"
            placeholder="请输入您的姓名（可选）"
            size="large"
            :prefix-icon="Edit"
          />
        </el-form-item>

```

```
35          <el-form-item>
36            <el-button
37              type="primary"
38              size="large"
39              :loading="loading"
40              @click="handleLogin"
41              class="login-btn"
42            >
43              {{ loading ? '登录中...' : '登录' }}
44            </el-button>
45          </el-form-item>
46        </el-form>
47
48        <div class="login-tips">
49          <p>💡 提示：开发阶段直接输入学号即可登录</p>
50          <p>生产环境将对接厦大统一认证系统</p>
51        </div>
52      </div>
53    </div>
54  </template>
55
56  <script setup>
57  import { ref, reactive } from 'vue'
58  import { useRouter } from 'vue-router'
59  import { useStore } from 'vuex'
60  import { ElMessage } from 'element-plus'
61  import { User, Edit } from '@element-plus/icons-vue'
62  import { authApi } from '@/api/auth'
63
64  const router = useRouter()
65  const store = useStore()
66
67  const loginFormRef = ref()
68  const loading = ref(false)
69
70  const loginForm = reactive({
71    schoolId: '',
72    username: ''
73  })
74
75  const loginRules = reactive({
76    schoolId: [
77      { required: true, message: '请输入学号/工号', trigger: 'blur' },
78      { min: 3, message: '学号长度至少3个字符', trigger: 'blur' }
79    ]
80  })
81
82  const handleLogin = async () => {
83    if (!loginFormRef.value) return
84
85    try {
86      const valid = await loginFormRef.value.validate()
87      if (!valid) return
88
```

```
    loading.value = true

    const { data } = await authApi.login(loginForm)

    // 保存用户信息和 token
    await store.dispatch('user/setUser', data.user)
    localStorage.setItem('token', data.token)

    ElMessage.success('登录成功')
    router.push('/')
  } catch (error) {
    console.error('登录失败:', error)
  } finally {
    loading.value = false
  }
}
</script>

<style scoped lang="less">
.login-container {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

.login-box {
  width: 400px;
  padding: 40px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
}

.login-header {
  text-align: center;
  margin-bottom: 30px;

  h1 {
    color: #333;
    margin-bottom: 8px;
    font-size: 24px;
  }

  p {
    color: #666;
    font-size: 14px;
  }
}

.login-form {
  .login-btn {
    width: 100%;
```

```
  }
}

.login-tips {
  margin-top: 20px;
  padding: 15px;
  background: #f8f9fa;
  border-radius: 4px;

  p {
    margin: 5px 0;
    font-size: 12px;
    color: #6c757d;
  }
}
</style>
```

动物列表页面组件

```
<!-- src/views/Animals.vue -->
<template>
  <div class="animals-page">
    <div class="page-header">
      <h1>🐱 校园动物档案</h1>
      <p>发现和了解厦门大学校园内可爱的动物朋友们</p>
    </div>

    <!-- 筛选条件 -->
    <div class="filter-section">
      <el-card shadow="never">
        <el-form :model="filters" inline>
          <el-form-item label="物种">
            <el-select v-model="filters.species" placeholder="全部物种" clearable>
              <el-option label="猫咪" value="cat" />
              <el-option label="狗狗" value="dog" />
              <el-option label="天鹅" value="swan" />
            </el-select>
          </el-form-item>

          <el-form-item label="区域">
            <el-input
              v-model="filters.area"
              placeholder="输入区域"
              clearable
            />
          </el-form-item>

          <el-form-item label="健康状态">
            <el-select v-model="filters.healthStatus" placeholder="全部状态" clearable>
              <el-option label="健康" value="healthy" />
              <el-option label="受伤" value="injured" />
              <el-option label="可领养" value="adoptable" />
            </el-select>
```

```
35          </el-form-item>
36
37          <el-form-item>
38            <el-button type="primary" @click="loadAnimals">搜索</el-button>
39            <el-button @click="resetFilters">重置</el-button>
40          </el-form-item>
41        </el-form>
42      </el-card>
43    </div>
44
45    <!-- 动物列表 -->
46    <div class="animals-grid">
47      <el-row :gutter="20">
48        <el-col
49          v-for="animal in animals"
50          :key="animal.animalId"
51          :xs="24"
52          :sm="12"
53          :md="8"
54          :lg="6"
55          class="animal-col"
56        >
57          <animal-card
58            :animal="animal"
59            @view-detail="viewAnimalDetail"
60            @report-sighting="showSightingDialog"
61          />
62        </el-col>
63      </el-row>
64    </div>
65
66    <!-- 分页 -->
67    <div class="pagination-section">
68      <el-pagination
69        v-model:current-page="pagination.current"
70        v-model:page-size="pagination.size"
71        :total="pagination.total"
72        :page-sizes="[8, 16, 24, 32]"
73        layout="total, sizes, prev, pager, next, jumper"
74        @size-change="handleSizeChange"
75        @current-change="handleCurrentChange"
76      />
77    </div>
78
79    <!-- 加载状态 -->
80    <el-empty v-if="!loading && animals.length === 0" description="暂无动物
数据" />
81    <el-skeleton v-if="loading" :rows="6" animated />
82
83    <!-- 目击记录对话框 -->
84    <sighting-dialog
85      v-model="sightingDialogVisible"
86      :animal-id="selectedAnimalId"
87      @success="handleSightingSuccess"
```

```
    />
  </div>
</template>

<script setup>
import { ref, reactive, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
import AnimalCard from '@/components/business/AnimalCard.vue'
import SightingDialog from '@/components/business/SightingDialog.vue'
import { animalsApi } from '@/api/animals'

const router = useRouter()

// 响应式数据
const animals = ref([])
const loading = ref(false)
const sightingDialogVisible = ref(false)
const selectedAnimalId = ref(null)

// 筛选条件
const filters = reactive({
  species: '',
  area: '',
  healthStatus: '',
  isMissing: false
})

// 分页配置
const pagination = reactive({
  current: 1,
  size: 8,
  total: 0
})

// 方法
const loadAnimals = async () => {
  loading.value = true
  try {
    const params = {
      ...filters,
      page: pagination.current - 1,
      size: pagination.size
    }

    const { data } = await animalsApi.getAnimals(params)
    animals.value = data.content
    pagination.total = data.totalElements
  } catch (error) {
    console.error('加载动物列表失败:', error)
    ElMessage.error('加载失败')
  } finally {
    loading.value = false
  }
```

```
142 }
143
144 const resetFilters = () => {
145   Object.keys(filters).forEach(key => {
146     filters[key] = ''
147   })
148   filters.isMissing = false
149   pagination.current = 1
150   loadAnimals()
151 }
152
153 const handleSizeChange = (size) => {
154   pagination.size = size
155   pagination.current = 1
156   loadAnimals()
157 }
158
159 const handleCurrentChange = (page) => {
160   pagination.current = page
161   loadAnimals()
162 }
163
164 const viewAnimalDetail = (animalId) => {
165   router.push(`/animals/${animalId}`)
166 }
167
168 const showSightingDialog = (animalId) => {
169   selectedAnimalId.value = animalId
170   sightingDialogVisible.value = true
171 }
172
173 const handleSightingSuccess = () => {
174   ElMessage.success('目击记录提交成功，等待管理员审核')
175   sightingDialogVisible.value = false
176 }
177
178 // 生命周期
179 onMounted(() => {
180   loadAnimals()
181 })
182 </script>
183
184 <style scoped lang="less">
185 .animals-page {
186   padding: 20px;
187   max-width: 1200px;
188   margin: 0 auto;
189 }
190
191 .page-header {
192   text-align: center;
193   margin-bottom: 30px;
194
195   h1 {
```

```
    color: #333;
    margin-bottom: 8px;
  }

  p {
    color: #666;
  }
}

.filter-section {
  margin-bottom: 20px;
}

.animals-grid {
  margin-bottom: 30px;
}

.animal-col {
  margin-bottom: 20px;
}

.pagination-section {
  display: flex;
  justify-content: center;
}
</style>
```

4. 集成路由和状态管理

Vuex 状态管理

```
// src/store/index.js
import { createStore } from 'vuex'
import user from './modules/user'
import app from './modules/app'

export default createStore({
  modules: {
    user,
    app
  }
})
```

```
// src/store/types.js
export const USER_SET_USER = 'USER_SET_USER'
export const USER_CLEAR_USER = 'USER_CLEAR_USER'
export const USER_SET_TOKEN = 'USER_SET_TOKEN'
export const USER_CLEAR_TOKEN = 'USER_CLEAR_TOKEN'

export const APP_SET_LOADING = 'APP_SET_LOADING'
export const APP_SET_ERROR = 'APP_SET_ERROR'
```

```
// src/store/modules/user.js
import { USER_SET_USER, USER_CLEAR_USER, USER_SET_TOKEN, USER_CLEAR_TOKEN }
from '../types'
```

```

const state = {
  user: null,
  token: localStorage.getItem('token') || null
}

const getters = {
  isLoggedIn: state => !!state.token,
  userInfo: state => state.user,
  userRole: state => state.user?.role,
  isAdmin: state => state.user?.role === 'ADMIN'
}

const mutations = {
  [USER_SET_USER](state, user) {
    state.user = user
  },

  [USER_CLEAR_USER](state) {
    state.user = null
  },

  [USER_SET_TOKEN](state, token) {
    state.token = token
    localStorage.setItem('token', token)
  },

  [USER_CLEAR_TOKEN](state) {
    state.token = null
    localStorage.removeItem('token')
  }
}

const actions = {
  setUser({ commit }, user) {
    commit(USER_SET_USER, user)
    localStorage.setItem('user', JSON.stringify(user))
  },

  setToken({ commit }, token) {
    commit(USER_SET_TOKEN, token)
  },

  logout({ commit }) {
    commit(USER_CLEAR_USER)
    commit(USER_CLEAR_TOKEN)
    localStorage.removeItem('user')
  },

  // 初始化用户状态
  initUser({ commit }) {
    const token = localStorage.getItem('token')
    const userStr = localStorage.getItem('user')

```

```
    if (token) {
      commit(USER_SET_TOKEN, token)
    }

    if (userStr) {
      try {
        const user = JSON.parse(userStr)
        commit(USER_SET_USER, user)
      } catch (error) {
        console.error('解析用户信息失败:', error)
        localStorage.removeItem('user')
      }
    }
  }
}

export default {
  namespaced: true,
  state,
  getters,
  mutations,
  actions
}
```

```
// src/store/modules/app.js
const state = {
  loading: false,
  error: null
}

const mutations = {
  SET_LOADING(state, loading) {
    state.loading = loading
  },

  SET_ERROR(state, error) {
    state.error = error
  }
}

const actions = {
  setLoading({ commit }, loading) {
    commit('SET_LOADING', loading)
  },

  setError({ commit }, error) {
    commit('SET_ERROR', error)
  },

  clearError({ commit }) {
    commit('SET_ERROR', null)
  }
}

export default {
```

```
  namespaced: true,
  state,
  mutations,
  actions
}
```

路由配置

```
// src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import { ElMessage } from 'element-plus'

const routes = [
  {
    path: '/login',
    name: 'Login',
    component: () => import('@/views/Login.vue'),
    meta: {
      title: '登录 - XMU动物友伴',
      requiresAuth: false
    }
  },
  {
    path: '/',
    name: 'Home',
    component: () => import('@/views/Home.vue'),
    meta: {
      title: '首页 - XMU动物友伴',
      requiresAuth: true
    }
  },
  {
    path: '/animals',
    name: 'Animals',
    component: () => import('@/views/Animals.vue'),
    meta: {
      title: '动物档案 - XMU动物友伴',
      requiresAuth: true
    }
  },
  {
    path: '/animals/:id',
    name: 'AnimalDetail',
    component: () => import('@/views/AnimalDetail.vue'),
    meta: {
      title: '动物详情 - XMU动物友伴',
      requiresAuth: true
    }
  },
  {
    path: '/sightings',
    name: 'Sightings',
    component: () => import('@/views/Sightings.vue'),
    meta: {
      title: '目击记录 - XMU动物友伴',
```

```
      requiresAuth: true
    }
  },
  {
    path: '/profile',
    name: 'Profile',
    component: () => import('@/views/Profile.vue'),
    meta: {
      title: '个人中心 - XMU动物友伴',
      requiresAuth: true
    }
  },
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: () => import('@/views/404.vue'),
    meta: {
      title: '页面未找到',
      requiresAuth: false
    }
  }
]

const router = createRouter({
  history: createWebHistory(process.env.BASE_URL),
  routes
})

// 路由守卫
router.beforeEach((to, from, next) => {
  // 设置页面标题
  if (to.meta.title) {
    document.title = to.meta.title
  }

  // 检查是否需要认证
  if (to.meta.requiresAuth) {
    const token = localStorage.getItem('token')
    if (!token) {
      ElMessage.warning('请先登录')
      next('/login')
      return
    }
  }

  // 如果已登录且访问登录页，重定向到首页
  if (to.path === '/login') {
    const token = localStorage.getItem('token')
    if (token) {
      next('/')
      return
    }
  }

```

```
  next()
})

// 路由错误处理
router.onError((error) => {
  console.error('路由错误:', error)
  ElMessage.error('页面加载失败')
})

export default router
```

主布局组件

```
<!-- src/components/layout/MainLayout.vue -->
<template>
  <div class="main-layout">
    <header class="layout-header">
      <nav-bar />
    </header>

    <main class="layout-main">
      <router-view />
    </main>

    <footer class="layout-footer">
      <p>🐾 XMU动物友伴 - 厦门大学校园动物管理系统</p>
    </footer>
  </div>
</template>

<script setup>
import NavBar from '@/components/layout/NavBar.vue'
</script>

<style scoped lang="less">
.main-layout {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}

.layout-header {
  position: sticky;
  top: 0;
  z-index: 1000;
}

.layout-main {
  flex: 1;
  padding-top: 0;
}

.layout-footer {
  background: #f8f9fa;
  padding: 20px;
```

```
  text-align: center;
  color: #6c757d;
  border-top: 1px solid #e9ecef;
}
</style>
```

这样，我们就完成了 Vue 前端项目的完整搭建。这个结构具有良好的可扩展性和维护性，支持模块化开发，集成了路由管理、状态管理和 API 服务层。

我来详细讲解阶段3的后端开发完整流程：

1. 设计数据库表结构

数据库设计原则

```sql
-- 创建数据库
CREATE DATABASE xmu_animal_companion CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE xmu_animal_companion;

-- 用户表
CREATE TABLE users (
    user_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL,
    school_id VARCHAR(50) NOT NULL UNIQUE,
    role ENUM('USER', 'ADMIN') DEFAULT 'USER',
    avatar_url VARCHAR(500),
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_school_id (school_id),
    INDEX idx_created_time (created_time)
);

-- 动物表
CREATE TABLE animals (
    animal_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    species ENUM('CAT', 'DOG', 'SWAN') NOT NULL,
    gender ENUM('MALE', 'FEMALE', 'UNKNOWN') DEFAULT 'UNKNOWN',
    area VARCHAR(200),
    appearance TEXT,
    personality TEXT,
    health_status ENUM('HEALTHY', 'INJURED', 'ADOPTABLE') DEFAULT 'HEALTHY',
    is_neutered BOOLEAN DEFAULT FALSE,
    is_missing BOOLEAN DEFAULT FALSE,
    avatar_url VARCHAR(500),
    last_seen_time TIMESTAMP NULL,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```sql
    INDEX idx_species (species),
    INDEX idx_area (area),
    INDEX idx_health_status (health_status),
    INDEX idx_is_missing (is_missing),
    INDEX idx_created_time (created_time)
);

-- 动物照片表
CREATE TABLE animal_photos (
    photo_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    animal_id BIGINT NOT NULL,
    photo_url VARCHAR(500) NOT NULL,
    display_order INT DEFAULT 0,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
    INDEX idx_animal_id (animal_id)
);

-- 动物性格标签表
CREATE TABLE animal_personality_tags (
    tag_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    animal_id BIGINT NOT NULL,
    tag_name VARCHAR(50) NOT NULL,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
    UNIQUE KEY uk_animal_tag (animal_id, tag_name),
    INDEX idx_animal_id (animal_id)
);

-- 目击记录表
CREATE TABLE sightings (
    sighting_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    animal_id BIGINT NOT NULL,
    user_id BIGINT NOT NULL,
    sighting_location VARCHAR(200) NOT NULL,
    latitude DECIMAL(10, 6),
    longitude DECIMAL(10, 6),
    description TEXT,
    status ENUM('PENDING', 'APPROVED', 'REJECTED') DEFAULT 'PENDING',
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    INDEX idx_animal_id (animal_id),
    INDEX idx_user_id (user_id),
    INDEX idx_status (status),
    INDEX idx_created_time (created_time)
);

-- 目击记录照片表
```

```sql
CREATE TABLE sighting_photos (
    photo_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    sighting_id BIGINT NOT NULL,
    photo_url VARCHAR(500) NOT NULL,
    display_order INT DEFAULT 0,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sighting_id) REFERENCES sightings(sighting_id) ON DELETE CASCADE,
    INDEX idx_sighting_id (sighting_id)
);

-- 收藏表
CREATE TABLE favorites (
    favorite_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    animal_id BIGINT NOT NULL,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
    UNIQUE KEY uk_user_animal (user_id, animal_id),
    INDEX idx_user_id (user_id),
    INDEX idx_animal_id (animal_id)
);

-- 失联报告表
CREATE TABLE missing_reports (
    report_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    animal_id BIGINT NOT NULL,
    user_id BIGINT NOT NULL,
    last_seen_time TIMESTAMP NOT NULL,
    last_seen_location VARCHAR(200) NOT NULL,
    report_note TEXT,
    status ENUM('SEARCHING', 'FOUND') DEFAULT 'SEARCHING',
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    INDEX idx_animal_id (animal_id),
    INDEX idx_user_id (user_id),
    INDEX idx_status (status),
    INDEX idx_created_time (created_time)
);

-- 插入初始数据
INSERT INTO users (username, school_id, role) VALUES
('管理员', 'admin001', 'ADMIN'),
('测试用户', '2123001', 'USER');

INSERT INTO animals (name, species, gender, area, appearance, personality, health_status, is_neutered) VALUES
```

```sql
('小橘', 'CAT', 'MALE', '芙蓉湖', '橘白相间，尾巴有黑色环纹', '性格温顺，喜欢被摸头', 'HEALTHY', TRUE),
('小黑', 'DOG', 'FEMALE', '海韵园', '全身黑色，中等体型', '活泼好动，喜欢和人玩耍', 'HEALTHY', FALSE),
('天鹅01', 'SWAN', 'UNKNOWN', '芙蓉湖', '全身白羽，优雅美丽', '优雅安静，常在湖中游弋', 'HEALTHY', FALSE);
```

2. 实现实体类和 Repository

Maven 依赖配置

```xml
<!-- pom.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<project>
    <dependencies>
        <!-- Spring Boot Starters -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- 数据库 -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- JWT -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.11.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
```

```
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.11.5</version>
            <scope>runtime</scope>
        </dependency>

        <!-- 文档 -->
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>2.3.0</version>
        </dependency>

        <!-- 工具 -->
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <!-- 测试 -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
</project>
```

应用配置

```yaml
# src/main/resources/application.yml
spring:
  application:
    name: xmu-animal-companion
  datasource:
    url: jdbc:mysql://localhost:3306/xmu_animal_companion?useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai
    username: root
    password: password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
```

```yaml
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true
  servlet:
    multipart:
      max-file-size: 10MB
      max-request-size: 10MB

server:
  port: 8080
  servlet:
    context-path: /

# JWT 配置
jwt:
  secret: xmu-animal-companion-secret-key-2025
  expiration: 86400000 # 24小时

# 文件上传配置
file:
  upload-dir: ./uploads
  access-url: http://localhost:8080/uploads/

# Swagger 配置
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
    operations-sorter: method
    tags-sorter: alpha

logging:
  level:
    com.xmu.animal: DEBUG
    org.springframework.security: DEBUG
```

枚举类定义

```java
// src/main/java/com/xmu/animal/enums/Species.java
package com.xmu.animal.enums;

public enum Species {
    CAT("猫咪"),
    DOG("狗狗"),
    SWAN("天鹅");

    private final String description;

    Species(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
```

```java
    }
}

// src/main/java/com/xmu/animal/enums/Gender.java
package com.xmu.animal.enums;

public enum Gender {
    MALE("雄性"),
    FEMALE("雌性"),
    UNKNOWN("未知");

    private final String description;

    Gender(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}

// src/main/java/com/xmu/animal/enums/HealthStatus.java
package com.xmu.animal.enums;

public enum HealthStatus {
    HEALTHY("健康"),
    INJURED("受伤"),
    ADOPTABLE("可领养");

    private final String description;

    HealthStatus(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}

// src/main/java/com/xmu/animal/enums/SightingStatus.java
package com.xmu.animal.enums;

public enum SightingStatus {
    PENDING("待审核"),
    APPROVED("已通过"),
    REJECTED("已拒绝");

    private final String description;

    SightingStatus(String description) {
        this.description = description;
    }
```

```java

    public String getDescription() {
        return description;
    }
}

// src/main/java/com/xmu/animal/enums/Role.java
package com.xmu.animal.enums;

public enum Role {
    USER("普通用户"),
    ADMIN("管理员");

    private final String description;

    Role(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}
```

实体类实现

```java
// src/main/java/com/xmu/animal/entity/User.java
package com.xmu.animal.entity;

import com.xmu.animal.enums.Role;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import javax.persistence.*;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;

    @Column(nullable = false, length = 100)
    private String username;

    @Column(nullable = false, unique = true, length = 50)
    private String schoolId;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role = Role.USER;

```

```java
    @Column(length = 500)
    private String avatarUrl;

    @CreationTimestamp
    private LocalDateTime createdTime;

    @UpdateTimestamp
    private LocalDateTime updatedTime;
}
```

```java
// src/main/java/com/xmu/animal/entity/Animal.java
package com.xmu.animal.entity;

import com.xmu.animal.enums.Species;
import com.xmu.animal.enums.Gender;
import com.xmu.animal.enums.HealthStatus;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import javax.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Data
@Entity
@Table(name = "animals")
public class Animal {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long animalId;

    @Column(nullable = false, length = 100)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Species species;

    @Enumerated(EnumType.STRING)
    private Gender gender = Gender.UNKNOWN;

    @Column(length = 200)
    private String area;

    @Column(columnDefinition = "TEXT")
    private String appearance;

    @Column(columnDefinition = "TEXT")
    private String personality;

    @Enumerated(EnumType.STRING)
    private HealthStatus healthStatus = HealthStatus.HEALTHY;

```

```java
    @Column(nullable = false)
    private Boolean isNeutered = false;

    @Column(nullable = false)
    private Boolean isMissing = false;

    @Column(length = 500)
    private String avatarUrl;

    @ElementCollection
    @CollectionTable(name = "animal_photos", joinColumns = @JoinColumn(name = "animal_id"))
    @Column(name = "photo_url")
    @OrderColumn(name = "display_order")
    private List<String> photoUrls = new ArrayList<>();

    @ElementCollection
    @CollectionTable(name = "animal_personality_tags", joinColumns = @JoinColumn(name = "animal_id"))
    @Column(name = "tag_name")
    private List<String> personalityTags = new ArrayList<>();

    private LocalDateTime lastSeenTime;

    @CreationTimestamp
    private LocalDateTime createdTime;

    @UpdateTimestamp
    private LocalDateTime updatedTime;
}
```

```java
// src/main/java/com/xmu/animal/entity/Sighting.java
package com.xmu.animal.entity;

import com.xmu.animal.enums.SightingStatus;
import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import javax.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Data
@Entity
@Table(name = "sightings")
public class Sighting {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long sightingId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "animal_id", nullable = false)
    private Animal animal;
```

```java

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false, length = 200)
    private String sightingLocation;

    private Double latitude;

    private Double longitude;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private SightingStatus status = SightingStatus.PENDING;

    @ElementCollection
    @CollectionTable(name = "sighting_photos", joinColumns = @JoinColumn(name = "sighting_id"))
    @Column(name = "photo_url")
    @OrderColumn(name = "display_order")
    private List<String> imageUrls = new ArrayList<>();

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;
}
```

```java
// src/main/java/com/xmu/animal/entity/Favorite.java
package com.xmu.animal.entity;

import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;

import javax.persistence.*;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "favorites", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"user_id", "animal_id"})
})
public class Favorite {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long favoriteId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;
```

```java

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "animal_id", nullable = false)
    private Animal animal;

    @CreationTimestamp
    private LocalDateTime createdTime;
}
```

```java
// src/main/java/com/xmu/animal/entity/MissingReport.java
package com.xmu.animal.entity;

import lombok.Data;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import javax.persistence.*;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "missing_reports")
public class MissingReport {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long reportId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "animal_id", nullable = false)
    private Animal animal;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false)
    private LocalDateTime lastSeenTime;

    @Column(nullable = false, length = 200)
    private String lastSeenLocation;

    @Column(columnDefinition = "TEXT")
    private String reportNote;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private ReportStatus status = ReportStatus.SEARCHING;

    @CreationTimestamp
    private LocalDateTime createdTime;

    @UpdateTimestamp
    private LocalDateTime updatedTime;

    public enum ReportStatus {
```

```
        SEARCHING, FOUND
    }
}
```

Repository 接口

```java
// src/main/java/com/xmu/animal/repository/UserRepository.java
package com.xmu.animal.repository;

import com.xmu.animal.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findBySchoolId(String schoolId);

    boolean existsBySchoolId(String schoolId);

    @Query("SELECT u FROM User u WHERE u.userId = :userId AND u.role = 'ADMIN'")
    Optional<User> findAdminById(@Param("userId") Long userId);
}
```

```java
// src/main/java/com/xmu/animal/repository/AnimalRepository.java
package com.xmu.animal.repository;

import com.xmu.animal.entity.Animal;
import com.xmu.animal.enums.HealthStatus;
import com.xmu.animal.enums.Species;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface AnimalRepository extends JpaRepository<Animal, Long>, JpaSpecificationExecutor<Animal> {

    Page<Animal> findBySpecies(Species species, Pageable pageable);

    Page<Animal> findByAreaContaining(String area, Pageable pageable);

```

```java
    Page<Animal> findByHealthStatus(HealthStatus healthStatus, Pageable pageable);

    Page<Animal> findByIsMissing(Boolean isMissing, Pageable pageable);

    List<Animal> findByNameContaining(String name);

    @Query("SELECT a FROM Animal a WHERE " +
           "(:species IS NULL OR a.species = :species) AND " +
           "(:area IS NULL OR a.area LIKE %:area%) AND " +
           "(:healthStatus IS NULL OR a.healthStatus = :healthStatus) AND " +
           "(:isMissing IS NULL OR a.isMissing = :isMissing)")
    Page<Animal> findByFilters(@Param("species") Species species,
                               @Param("area") String area,
                               @Param("healthStatus") HealthStatus healthStatus,
                               @Param("isMissing") Boolean isMissing,
                               Pageable pageable);

    @Query("SELECT a FROM Animal a JOIN Favorite f ON a.animalId = f.animal.animalId " +
           "WHERE f.user.userId = :userId ORDER BY f.createdTime DESC")
    Page<Animal> findFavoritesByUserId(@Param("userId") Long userId, Pageable pageable);

    Optional<Animal> findByNameAndArea(String name, String area);
}
```

```java
// src/main/java/com/xmu/animal/repository/SightingRepository.java
package com.xmu.animal.repository;

import com.xmu.animal.entity.Sighting;
import com.xmu.animal.enums.SightingStatus;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface SightingRepository extends JpaRepository<Sighting, Long> {

    Page<Sighting> findByStatus(SightingStatus status, Pageable pageable);

    Page<Sighting> findByUserId(Long userId, Pageable pageable);

    Page<Sighting> findByAnimalAnimalId(Long animalId, Pageable pageable);

    @Query("SELECT s FROM Sighting s WHERE s.status = 'APPROVED' ORDER BY s.createdAt DESC")
```

```java
    Page<Sighting> findApprovedSightings(Pageable pageable);

    @Query("SELECT s FROM Sighting s WHERE s.user.userId = :userId ORDER BY s.createdAt DESC")
    Page<Sighting> findByUserUserId(@Param("userId") Long userId, Pageable pageable);

    @Query("SELECT COUNT(s) FROM Sighting s WHERE s.animal.animalId = :animalId AND s.status = 'APPROVED'")
    Long countApprovedSightingsByAnimalId(@Param("animalId") Long animalId);

    List<Sighting> findByStatusAndAnimalAnimalId(SightingStatus status, Long animalId);
}
```

```java
// src/main/java/com/xmu/animal/repository/FavoriteRepository.java
package com.xmu.animal.repository;

import com.xmu.animal.entity.Favorite;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface FavoriteRepository extends JpaRepository<Favorite, Long> {

    Optional<Favorite> findByUserUserIdAndAnimalAnimalId(Long userId, Long animalId);

    Page<Favorite> findByUserUserId(Long userId, Pageable pageable);

    boolean existsByUserUserIdAndAnimalAnimalId(Long userId, Long animalId);

    @Query("SELECT COUNT(f) FROM Favorite f WHERE f.animal.animalId = :animalId")
    Long countByAnimalAnimalId(@Param("animalId") Long animalId);

    void deleteByUserUserIdAndAnimalAnimalId(Long userId, Long animalId);
}
```

3. 开发 Service 和 Controller

DTO（数据传输对象）

```java
// src/main/java/com/xmu/animal/dto/LoginRequest.java
package com.xmu.animal.dto;

import lombok.Data;
import javax.validation.constraints.NotBlank;

```

```java
@Data
public class LoginRequest {
    @NotBlank(message = "学号不能为空")
    private String schoolId;

    private String username;
}

// src/main/java/com/xmu/animal/dto/LoginResponse.java
package com.xmu.animal.dto;

import com.xmu.animal.entity.User;
import lombok.Data;

@Data
public class LoginResponse {
    private String token;
    private User user;

    public LoginResponse(String token, User user) {
        this.token = token;
        this.user = user;
    }
}

// src/main/java/com/xmu/animal/dto/AnimalRequest.java
package com.xmu.animal.dto;

import com.xmu.animal.enums.Species;
import com.xmu.animal.enums.Gender;
import com.xmu.animal.enums.HealthStatus;
import lombok.Data;
import javax.validation.constraints.NotBlank;
import java.util.List;

@Data
public class AnimalRequest {
    @NotBlank(message = "动物名称不能为空")
    private String name;

    private Species species;
    private Gender gender;
    private String area;
    private String appearance;
    private String personality;
    private HealthStatus healthStatus;
    private Boolean isNeutered = false;
    private List<String> personalityTags;
}

// src/main/java/com/xmu/animal/dto/SightingRequest.java
package com.xmu.animal.dto;

import lombok.Data;
```

```java
import javax.validation.constraints.NotNull;

@Data
public class SightingRequest {
    @NotNull(message = "动物ID不能为空")
    private Long animalId;

    @NotNull(message = "目击地点不能为空")
    private String sightingLocation;

    private Double latitude;
    private Double longitude;
    private String description;
    private List<String> imageUrls;
}
```

统一响应格式

```java
// src/main/java/com/xmu/animal/dto/ApiResponse.java
package com.xmu.animal.dto;

import lombok.Data;
import java.time.LocalDateTime;

@Data
public class ApiResponse<T> {
    private Integer code;
    private String message;
    private T data;
    private LocalDateTime timestamp;

    public ApiResponse(Integer code, String message, T data) {
        this.code = code;
        this.message = message;
        this.data = data;
        this.timestamp = LocalDateTime.now();
    }

    public static <T> ApiResponse<T> success(T data) {
        return new ApiResponse<>(200, "success", data);
    }

    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(200, message, data);
    }

    public static <T> ApiResponse<T> error(Integer code, String message) {
        return new ApiResponse<>(code, message, null);
    }

    public static <T> ApiResponse<T> created(T data) {
        return new ApiResponse<>(201, "created", data);
    }
}
```

Service 层实现

```java
// src/main/java/com/xmu/animal/service/UserService.java
package com.xmu.animal.service;

import com.xmu.animal.entity.User;
import com.xmu.animal.enums.Role;

public interface UserService {
    User findBySchoolId(String schoolId);
    User findById(Long userId);
    User createUser(String schoolId, String username, Role role);
    boolean existsBySchoolId(String schoolId);
    User getCurrentUser();
    Long getCurrentUserId();
}
```

```java
// src/main/java/com/xmu/animal/service/impl/UserServiceImpl.java
package com.xmu.animal.service.impl;

import com.xmu.animal.entity.User;
import com.xmu.animal.enums.Role;
import com.xmu.animal.repository.UserRepository;
import com.xmu.animal.service.UserService;
import lombok.RequiredArgsConstructor;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
public class UserServiceImpl implements UserService {

    private final UserRepository userRepository;

    @Override
    public User findBySchoolId(String schoolId) {
        return userRepository.findBySchoolId(schoolId)
                .orElseThrow(() -> new RuntimeException("用户不存在"));
    }

    @Override
    public User findById(Long userId) {
        return userRepository.findById(userId)
                .orElseThrow(() -> new RuntimeException("用户不存在"));
    }

    @Override
    @Transactional
    public User createUser(String schoolId, String username, Role role) {
        if (userRepository.existsBySchoolId(schoolId)) {
            throw new RuntimeException("学号已存在");
        }

        User user = new User();
```

```java
        user.setSchoolId(schoolId);
        user.setUsername(username != null ? username : "用户" + schoolId);
        user.setRole(role);

        return userRepository.save(user);
    }

    @Override
    public boolean existsBySchoolId(String schoolId) {
        return userRepository.existsBySchoolId(schoolId);
    }

    @Override
    public User getCurrentUser() {
        String schoolId = SecurityContextHolder.getContext().getAuthentication().getName();
        return findBySchoolId(schoolId);
    }

    @Override
    public Long getCurrentUserId() {
        return getCurrentUser().getUserId();
    }
}
```

```java
// src/main/java/com/xmu/animal/service/AnimalService.java
package com.xmu.animal.service;

import com.xmu.animal.dto.AnimalRequest;
import com.xmu.animal.entity.Animal;
import com.xmu.animal.enums.HealthStatus;
import com.xmu.animal.enums.Species;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;

public interface AnimalService {
    Page<Animal> getAnimals(Species species, String area, HealthStatus healthStatus,
                            Boolean isMissing, Pageable pageable);
    Animal getAnimalById(Long animalId);
    Animal createAnimal(AnimalRequest request);
    Animal updateAnimal(Long animalId, AnimalRequest request);
    void deleteAnimal(Long animalId);
    Page<Animal> getFavoriteAnimals(Long userId, Pageable pageable);
    void updateLastSeenTime(Long animalId);
}
```

```java
// src/main/java/com/xmu/animal/service/impl/AnimalServiceImpl.java
package com.xmu.animal.service.impl;

import com.xmu.animal.dto.AnimalRequest;
import com.xmu.animal.entity.Animal;
import com.xmu.animal.enums.HealthStatus;
import com.xmu.animal.enums.Species;
import com.xmu.animal.repository.AnimalRepository;
```

```java
import com.xmu.animal.service.AnimalService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

@Service
@RequiredArgsConstructor
public class AnimalServiceImpl implements AnimalService {

    private final AnimalRepository animalRepository;

    @Override
    public Page<Animal> getAnimals(Species species, String area, HealthStatus healthStatus,
                                   Boolean isMissing, Pageable pageable) {
        return animalRepository.findByFilters(species, area, healthStatus, isMissing, pageable);
    }

    @Override
    public Animal getAnimalById(Long animalId) {
        return animalRepository.findById(animalId)
                .orElseThrow(() -> new RuntimeException("动物不存在"));
    }

    @Override
    @Transactional
    public Animal createAnimal(AnimalRequest request) {
        // 检查同名动物在同一区域是否已存在
        if (request.getArea() != null) {
            animalRepository.findByNameAndArea(request.getName(), request.getArea())
                    .ifPresent(animal -> {
                        throw new RuntimeException("该区域已存在同名动物");
                    });
        }

        Animal animal = new Animal();
        updateAnimalFromRequest(animal, request);

        return animalRepository.save(animal);
    }

    @Override
    @Transactional
    public Animal updateAnimal(Long animalId, AnimalRequest request) {
        Animal animal = getAnimalById(animalId);
        updateAnimalFromRequest(animal, request);
        return animalRepository.save(animal);
    }
```

```java

    @Override
    @Transactional
    public void deleteAnimal(Long animalId) {
        Animal animal = getAnimalById(animalId);
        animalRepository.delete(animal);
    }

    @Override
    public Page<Animal> getFavoriteAnimals(Long userId, Pageable pageable) {
        return animalRepository.findFavoritesByUserId(userId, pageable);
    }

    @Override
    @Transactional
    public void updateLastSeenTime(Long animalId) {
        Animal animal = getAnimalById(animalId);
        animal.setLastSeenTime(LocalDateTime.now());
        animalRepository.save(animal);
    }

    private void updateAnimalFromRequest(Animal animal, AnimalRequest request) {
        animal.setName(request.getName());
        animal.setSpecies(request.getSpecies());
        animal.setGender(request.getGender());
        animal.setArea(request.getArea());
        animal.setAppearance(request.getAppearance());
        animal.setPersonality(request.getPersonality());
        animal.setHealthStatus(request.getHealthStatus());
        animal.setIsNeutered(request.getIsNeutered() != null ? request.getIsNeutered() : false);

        if (request.getPersonalityTags() != null) {
            animal.setPersonalityTags(request.getPersonalityTags());
        }
    }
}
```

Controller 层实现

```java
// src/main/java/com/xmu/animal/controller/AuthController.java
package com.xmu.animal.controller;

import com.xmu.animal.dto.ApiResponse;
import com.xmu.animal.dto.LoginRequest;
import com.xmu.animal.dto.LoginResponse;
import com.xmu.animal.entity.User;
import com.xmu.animal.enums.Role;
import com.xmu.animal.service.UserService;
import com.xmu.animal.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import javax.validation.Valid;
```

```java

@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final UserService userService;
    private final JwtUtil jwtUtil;

    @PostMapping("/login")
    public ApiResponse<LoginResponse> login(@Valid @RequestBody LoginRequest request) {
        User user;

        // 查找或创建用户
        if (userService.existsBySchoolId(request.getSchoolId())) {
            user = userService.findBySchoolId(request.getSchoolId());
        } else {
            // 第一个用户设为管理员，其他为普通用户
            Role role = userService.findById(1L) == null ? Role.ADMIN : Role.USER;
            user = userService.createUser(request.getSchoolId(), request.getUsername(), role);
        }

        // 生成 JWT token
        String token = jwtUtil.generateToken(user.getSchoolId(), user.getRole());

        LoginResponse response = new LoginResponse(token, user);
        return ApiResponse.success(response);
    }
}
```

```java
// src/main/java/com/xmu/animal/controller/AnimalController.java
package com.xmu.animal.controller;

import com.xmu.animal.dto.ApiResponse;
import com.xmu.animal.dto.AnimalRequest;
import com.xmu.animal.entity.Animal;
import com.xmu.animal.enums.HealthStatus;
import com.xmu.animal.enums.Species;
import com.xmu.animal.service.AnimalService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import javax.validation.Valid;

@RestController
@RequestMapping("/api/animals")
```

```java
@RequiredArgsConstructor
public class AnimalController {

    private final AnimalService animalService;

    @GetMapping
    public ApiResponse<Page<Animal>> getAnimals(
            @RequestParam(required = false) Species species,
            @RequestParam(required = false) String area,
            @RequestParam(required = false) HealthStatus healthStatus,
            @RequestParam(required = false) Boolean isMissing,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size) {

        Pageable pageable = PageRequest.of(page, size, Sort.by("createdTime").descending());
        Page<Animal> animals = animalService.getAnimals(species, area, healthStatus, isMissing, pageable);
        return ApiResponse.success(animals);
    }

    @GetMapping("/{id}")
    public ApiResponse<Animal> getAnimalDetail(@PathVariable Long id) {
        Animal animal = animalService.getAnimalById(id);
        return ApiResponse.success(animal);
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ApiResponse<Animal> createAnimal(@Valid @RequestBody AnimalRequest request) {
        Animal animal = animalService.createAnimal(request);
        return ApiResponse.created(animal);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ApiResponse<Animal> updateAnimal(@PathVariable Long id, @Valid @RequestBody AnimalRequest request) {
        Animal animal = animalService.updateAnimal(id, request);
        return ApiResponse.success(animal);
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ApiResponse<Void> deleteAnimal(@PathVariable Long id) {
        animalService.deleteAnimal(id);
        return ApiResponse.success(null);
    }
}
```

4. 配置安全和权限

JWT 工具类

```java
// src/main/java/com/xmu/animal/util/JwtUtil.java
package com.xmu.animal.util;

import com.xmu.animal.enums.Role;
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;

@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private Long expiration;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String schoolId, Role role) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + expiration);

        return Jwts.builder()
                .setSubject(schoolId)
                .claim("role", role.name())
                .setIssuedAt(now)
                .setExpiration(expiryDate)
                .signWith(getSigningKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    public String getSchoolIdFromToken(String token) {
        Claims claims = Jwts.parserBuilder()
                .setSigningKey(getSigningKey())
                .build()
                .parseClaimsJws(token)
                .getBody();

        return claims.getSubject();
    }

    public String getRoleFromToken(String token) {
        Claims claims = Jwts.parserBuilder()
```

```
                .setSigningKey(getSigningKey())
                .build()
                .parseClaimsJws(token)
                .getBody();

        return claims.get("role", String.class);
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder()
                    .setSigningKey(getSigningKey())
                    .build()
                    .parseClaimsJws(token);
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }
}
```

Spring Security 配置

```
// src/main/java/com/xmu/animal/config/SecurityConfig.java
package com.xmu.animal.config;

import com.xmu.animal.filter.JwtAuthenticationFilter;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.method.configuration.EnableGlobalMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(prePostEnabled = true)
@RequiredArgsConstructor
public class SecurityConfig {

```

```java
    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .cors().and()
            .csrf().disable()
            .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            .and()
            .authorizeRequests()
                .antMatchers("/api/auth/**").permitAll()
                .antMatchers("/api/upload").permitAll()
                .antMatchers("/uploads/**").permitAll()
                .antMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOriginPatterns(Arrays.asList("*"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(Arrays.asList("*"));
        configuration.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

JWT 认证过滤器

```java
// src/main/java/com/xmu/animal/filter/JwtAuthenticationFilter.java
package com.xmu.animal.filter;

import com.xmu.animal.util.JwtUtil;
import lombok.RequiredArgsConstructor;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
```

```java

import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Collections;

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String path = request.getServletPath();

        // 放行公开路径
        if (path.startsWith("/api/auth") ||
            path.startsWith("/uploads") ||
            path.startsWith("/swagger-ui") ||
            path.startsWith("/v3/api-docs")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = getTokenFromRequest(request);

        if (token != null && jwtUtil.validateToken(token)) {
            String schoolId = jwtUtil.getSchoolIdFromToken(token);
            String role = jwtUtil.getRoleFromToken(token);

            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(
                    schoolId,
                    null,
                    Collections.singletonList(new SimpleGrantedAuthority("ROLE_" + role))
                );


 SecurityContextHolder.getContext().setAuthentication(authentication);
        } else {
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.getWriter().write("{\"code\":401,\"message\":\"未授权\"}");
            return;
        }

```

```java
        filterChain.doFilter(request, response);
    }

    private String getTokenFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (bearerToken != null && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}
```

全局异常处理

```java
// src/main/java/com/xmu/animal/exception/GlobalExceptionHandler.java
package com.xmu.animal.exception;

import com.xmu.animal.dto.ApiResponse;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import javax.servlet.http.HttpServletRequest;

@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiResponse<Void> handleRuntimeException(RuntimeException e, HttpServletRequest request) {
        log.error("业务异常: {} - {}", request.getRequestURI(), e.getMessage());
        return ApiResponse.error(400, e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiResponse<Void> handleValidationException(MethodArgumentNotValidException e) {
        String message = e.getBindingResult().getFieldErrors().get(0).getDefaultMessage();
        return ApiResponse.error(400, message);
    }

    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public ApiResponse<Void> handleAccessDeniedException(AccessDeniedException e) {
        return ApiResponse.error(403, "权限不足");
```

```java
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ApiResponse<Void> handleException(Exception e, HttpServletRequest request) {
        log.error("系统异常: {} - {}", request.getRequestURI(), e.getMessage(), e);
        return ApiResponse.error(500, "服务器内部错误");
    }
}
```

文件上传配置

```java
// src/main/java/com/xmu/animal/config/FileUploadConfig.java
package com.xmu.animal.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import java.nio.file.Paths;

@Configuration
public class FileUploadConfig implements WebMvcConfigurer {

    @Value("${file.upload-dir}")
    private String uploadDir;

    @Value("${file.access-url}")
    private String accessUrl;

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        String uploadPath = Paths.get(uploadDir).toAbsolutePath().toString();

        registry.addResourceHandler("/uploads/**")
                .addResourceLocations("file:" + uploadPath + "/");
    }
}
```

主应用类

```java
// src/main/java/com/xmu/animal/XmuAnimalCompanionApplication.java
package com.xmu.animal;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@SpringBootApplication
@EnableJpaAuditing
```

```java
public class XmuAnimalCompanionApplication {

    public static void main(String[] args) {
        SpringApplication.run(XmuAnimalCompanionApplication.class, args);
    }
}
```

这样我们就完成了完整的后端开发，包括数据库设计、实体类、Repository、Service、Controller 以及安全配置。系统具备了完整的认证授权、数据验证、异常处理和文件上传功能。